

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине : «Модели решения задач в ИС»

По теме : «Классификация линейно разделимых
образов однослойным персептроном для
логических функций n переменных»

Выполнил:

студент 3 курса
группы ИИ-26
Заруцкий В.Я.

Проверил:

Адренко К.В.

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Вариант 4:

№ of variant	n	Task
4	5	AND ⁱ

Ход работы :

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import product

n = 5
task = "NAND"

def logic_function(x):
    if task == "NAND":
        return int(not all(x))
    raise ValueError("Unknown task")

X = np.array(list(product([0, 1], repeat=n)))
y = np.array([logic_function(x) for x in X]).reshape(-1, 1)

np.random.seed(1)
indices = np.random.permutation(len(X))
train_size = len(X) // 2

train_idx = indices[:train_size]
test_idx = indices[train_size:]

X_train, y_train = X[train_idx], y[train_idx]
X_test, y_test = X[test_idx], y[test_idx]

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

def train_perceptron(X, y, alpha=0.1, adaptive=False, max_epochs=5000):
    w = np.random.uniform(-1, 1, (n, 1))
    b = np.random.uniform(-1, 1)

    errors_train = []
    errors_test = []

    for epoch in range(max_epochs):
        net = X @ w + b
        out = sigmoid(net)

        error = y - out
        mse = np.mean(error ** 2)
        errors_train.append(mse)
        test_out = sigmoid(X_test @ w + b)
        test_mse = np.mean((y_test - test_out) ** 2)
        errors_test.append(test_mse)

        grad = error * sigmoid_derivative(out)

        if adaptive:
            alpha = 0.1 / (1 + 0.01 * epoch)

        w += alpha * (X.T @ grad)
        b += alpha * np.sum(grad)

        if mse < 1e-4:
            break

    return w, b, errors_train, errors_test, epoch + 1

w_fixed, b_fixed, err_train_fixed, err_test_fixed, epochs_fixed = train_perceptron(
    X_train, y_train, alpha=0.1, adaptive=False
)
```

```

w_adapt, b_adapt, err_train_adapt, err_test_adapt, epochs_adapt = train_perceptron(
    X_train, y_train, alpha=0.1, adaptive=True
)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.plot(range(1, len(err_train_fixed) + 1), err_train_fixed, 'b-o',
         label="Train (фиксированный  $\alpha$ ", linewidth=2, markersize=6)
plt.plot(range(1, len(err_test_fixed) + 1), err_test_fixed, 'r--s',
         label="Test (фиксированный  $\alpha$ ", linewidth=2, markersize=6)
plt.title("Фиксированный шаг обучения")
plt.xlabel("Эпоха")
plt.ylabel("Ошибка (MSE)")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(range(1, len(err_train_adapt) + 1), err_train_adapt, 'b-s',
         label="Train (адаптивный  $\alpha$ ", linewidth=2, markersize=6)
plt.plot(range(1, len(err_test_adapt) + 1), err_test_adapt, 'r-.^',
         label="Test (адаптивный  $\alpha$ ", linewidth=2, markersize=6)
plt.title("Адаптивный шаг обучения")
plt.xlabel("Эпоха")
plt.ylabel("Ошибка (MSE)")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

def predict(x, w, b):
    prob = sigmoid(np.dot(x, w) + b)[0]
    return prob, int(prob >= 0.5)

print("\n=== РЕЗУЛЬТАТЫ ОБУЧЕНИЯ (NAND) ===")
print(f"Эпох (фиксированный шаг): {epochs_fixed}")
print(f"Эпох (адаптивный шаг): {epochs_adapt}")

print("\nВесовые коэффициенты (фиксированный шаг):")
print(w_fixed.flatten())
print("Попор:", b_fixed)

print("\nВесовые коэффициенты (адаптивный шаг):")
print(w_adapt.flatten())
print("Попор:", b_adapt)

print("\n=== Проверка на полной таблице истинности ===")
for x in X:
    prob, cls = predict(x, w_adapt, b_adapt)
    print(f"{x} → P={prob:.3f}, class={cls}")

```

Результат тестирования:

=== РЕЗУЛЬТАТЫ ОБУЧЕНИЯ (NAND) ===

Эпох (фиксированный шаг): 2308

Эпох (адаптивный шаг): 5000

Весовые коэффициенты (фиксированный шаг):

[1.74574432 0.56700531 1.64152192 1.95259008 1.05325438]

Порог: 1.9539708862316685

Весовые коэффициенты (адаптивный шаг):

[0.36738607 0.22759597 1.09190771 0.51711013 0.47115727]

Порог: 2.7804603384979236

=== Проверка на полной таблице истинности ===

[0 0 0 0] → P=0.942, class=1

[0 0 0 1] → P=0.963, class=1

[0 0 0 1 0] → P=0.964, class=1

[0 0 0 1 1] → P=0.977, class=1

[0 0 1 0 0] → P=0.980, class=1

[0 0 1 0 1] → P=0.987, class=1

[0 0 1 1 0] → P=0.988, class=1

[0 0 1 1 1] → P=0.992, class=1

[0 1 0 0 0] → P=0.953, class=1

[0 1 0 0 1] → P=0.970, class=1

[0 1 0 1 0] → P=0.971, class=1

[0 1 0 1 1] → P=0.982, class=1

[0 1 1 0 0] → P=0.984, class=1

[0 1 1 0 1] → P=0.990, class=1

[0 1 1 1 0] → P=0.990, class=1

[0 1 1 1 1] → P=0.994, class=1

[1 0 0 0 0] → P=0.959, class=1

[1 0 0 0 1] → P=0.974, class=1

[1 0 0 1 0] → P=0.975, class=1

[1 0 0 1 1] → P=0.984, class=1

[1 0 1 0 0] → P=0.986, class=1

[1 0 1 0 1] → P=0.991, class=1

[1 0 1 1 0] → P=0.991, class=1

[1 0 1 1 1] → P=0.995, class=1

[1 1 0 0 0] → P=0.967, class=1

[1 1 0 0 1] → P=0.979, class=1

[1 1 0 1 0] → P=0.980, class=1

[1 1 0 1 1] → P=0.987, class=1

[1 1 1 0 0] → P=0.989, class=1

[1 1 1 0 1] → P=0.993, class=1

[1 1 1 1 0] → P=0.993, class=1

[1 1 1 1 1] → P=0.996, class=1

График Фиксированного шага обучения:

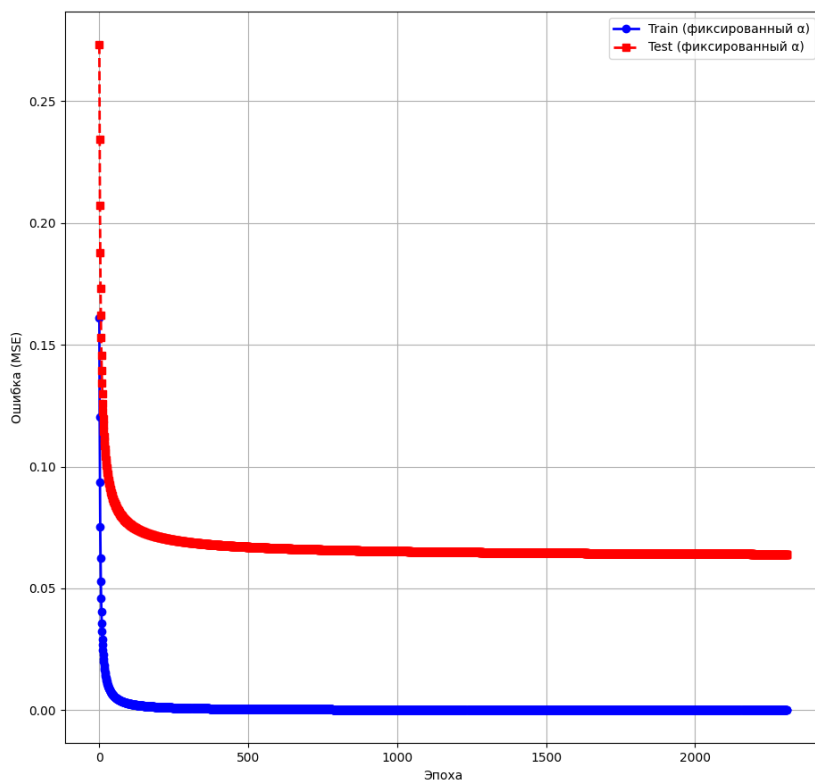
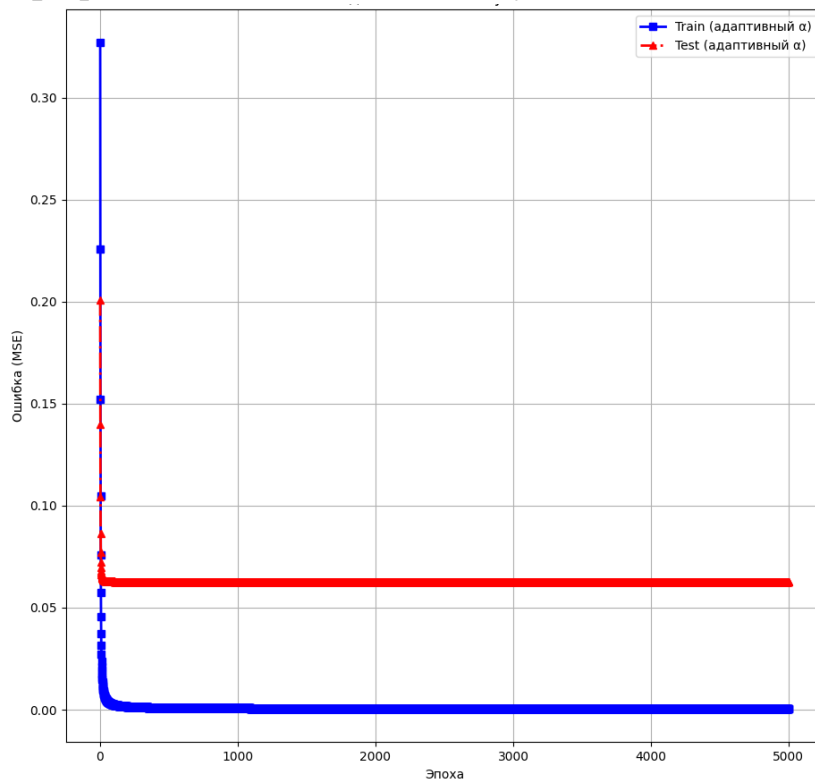


График Адаптивного шага обучения:



Вывод: Реализовал нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнил эффективность обучения при различных начальных условиях и параметрах шага.